

Algorytmy ewolucyjne

Zasada działania, obszary zastosowań, biblioteki programistyczne, zastosowanie do wybranego problemu optymalizacyjnego oraz eksperymenty

Marcin Habuda, Illia Tarasenko

1. Zasada działania i zastosowania

Zasada działania algorytmu genetycznego (GA)

Algorytm genetyczny to metoda optymalizacji inspirowana **ewolucją biologiczną**. Szuka najlepszego rozwiązania przez symulowanie procesów takich jak **dobór naturalny**, **krzyżowanie** i **mutacja**.

Kroki działania GA:

1. **Inicjalizacja populacji** - losowo generuje zbiór początkowych rozwiązań (osobników).
2. **Ocena (fitness)** - dla każdego osobnika obliczana jest wartość funkcji celu, określająca jakość rozwiązania.
3. **Selekcja** - wybierane są lepsze osobniki, które będą „rodzicami”.
4. **Krzyżowanie (crossover)** - rodzice są łączeni, by tworzyć nowe rozwiązania.
5. **Mutacja** - niektóre rozwiązania są losowo modyfikowane, aby zwiększyć różnorodność.
6. **Zastąpienie populacji** - nowa generacja osobników zastępuje starą.
7. **Powtarzanie** - proces trwa, aż zostanie osiągnięte dobre rozwiązanie lub limit iteracji.

Obszary zastosowań algorytmów genetycznych

- Optymalizacja funkcji i parametrów w złożonych przestrzeniach rozwiązań.
- Projektowanie i inżynieria konstrukcji.
- Sztuczna inteligencja w grach i strategiach.
- Planowanie i optymalizacja tras (np. problem komiwojażera).
- Dobór parametrów i uczenie modeli w systemach AI.
- Bioinformatyka i analiza sekwencji genetycznych.

Algorytm genetyczny dobrze nadaje się do naszego problemu, ponieważ debiuty szachowe to sekwencje **dyskretne**, czyli konkretne ruchy figur, a nie wartości liczbowe. GA potrafi efektywnie operować na takich sekwencjach - łatwo je kodować jako chromosomy, łączyć przez crossover i modyfikować przez mutację. Inne algorytmy, jak np. strategie ewolucyjne, są lepsze dla zmiennych ciągłych, więc w naszym przypadku GA był naturalnym wyborem.

Celem projektu jest zastosowanie **algorytmu ewolucyjnego** do **automatycznego generowania i optymalizacji debiutów szachowych**. Algorytm analizuje pierwsze ruchy w partii i stara się znaleźć takie sekwencje debiutowe, które dają jak największą przewagę białemu graczowi.

[ChessOpeningGA/main.py at master · RedSSD/ChessOpeningGA · GitHub](#)

Podany wyżej algorytm ewolucyjny opisuje debiuty partii szachowych. Algorytm generuje 3 pierwsze możliwe ruchy i sprawdza dla jednego z graczy sukces każdego z nich na podstawie podanych wag danych figur szachowych. Losowa wymiana pojedynczych genów z debiutów pozwala generować nowe potencjalnie lepsze zagrania.

1. Tworzy populację debiutów szachowych i statystykę każdego możliwego debiutu na szachownicy. Pojedyncze chromosomy odpowiadają konkretnemu ruchowi na szachownicy.
2. Dla każdego debiutu z populacji wylicza fitness i tworzy mapę debiutów z odpowiadającymi im fitnessami.
3. Następnie tworzy potomstwo z populacji debiutów. Dla potomnej populacji robi crossover z prawdopodobieństwem 70%. Tworzy nowego potomka z nową wartością fitness, która jest wyliczana na podstawie funkcji mate. Funkcja mate wylicza fitness na podstawie score, który jest obliczany przez silnik szachowy; najwyższa wartość fitness równa 1 jest gdy biały gracz zrobi mata, najniższa natomiast wartość -1 jest gdy czarny gracz zrobi mata.
4. Każdy potomek może zostać losowo zmutowany z prawdopodobieństwem 20%, czyli obecny ruch może zostać wymieniony na nowy losowy ruch z listy możliwych ruchów. W ten sposób algorytm tworzy nowy debiut, dla którego znów wyliczany jest fitness.

2. Biblioteki programistyczne

Biblioteka / Silnik	Instalacja	Możliwości	Dostępne algorytmy / funkcje	Zalety	Wady
DEAP (Python)	<code>pip install deap==1.4.3</code>	Tworzenie populacji, selekcja, krzyżowanie, mutacja, statystyki	GA, ES, GP, koewolucja	Elastyczna, łatwa integracja, multiprocessing	Brak gotowych funkcji dla specyficznych problemów
Genetics (Java)	Maven dependency <dependency> io.jenetics:je- netics:7.1.0< /dependency>	GA, ES, multi- objective GA, wizualizacja	GA, ES, multi- objective, koewolucja	Silna typizacja, dobra modularność	Wolniejsze prototypowanie niż Python
python-chess (Python)	<code>pip install python- chess==1.999</code>	Reprezentacja szachownicy, generowanie ruchów, PGN/FEN	-	Prosty interfejs, dobra dokumentacja	Brak AI, wymaga silnika
Stockfish (Silnik UCI)	<code>sudo apt install stockfish</code>	Ocena pozycji, generowanie ruchów, analiza wariantów	Minimax, alfa-beta, heurystyki	Bardzo silny, szybki	Nie generuje strategii, tylko ocenia
PyGAD (Python)	<code>pip install pygad</code>	Łatwy GA, wsparcie dla NN, optymalizacja parametrów	GA	Bardzo prosty w użyciu	Ograniczona konfigurowalność
NEAT-Python	<code>pip install neat-python</code>	Ewolucja sieci neuronowych (NEAT)	NEAT	Automatyczna topologia NN, dobre przykłady	Trudniejsze debugowanie
TPOT (Python)	<code>pip install tpot</code>	AutoML, optymalizacja pipeline ML	GA w AutoML	Automatyczne generowanie modeli ML	Bardzo wolne przy dużych danych
EvoPy (Python)	<code>pip install evopy</code>	Minimalistyczna biblioteka GA	GA, prosty ES	Bardzo lekka	Małe możliwości
OpenSpiel (C++/Python)	<code>pip install open_spiel</code>	Gry planszowe, teoria gier, RL	CFR, MCTS, Minimax, RL	Obsługa wielu gier	Wymaga wiedzy o RL
Leela Chess Zero (Lichess- Net NN)	pobranie binariów Lc0	Silnik oparty na NN	Monte Carlo Tree Search + NN	Bardzo silne AI, self-play	Wymaga GPU, trudne w użyciu

DEAP - opis szczegółowy

DEAP (Distributed Evolutionary Algorithms in Python) to elastyczna i rozbudowana biblioteka przeznaczona do tworzenia, uruchamiania i analizowania algorytmów ewolucyjnych w języku Python. Biblioteka jest podzielona na kilka kluczowych modułów:

- **creator** - służy do definiowania struktur osobników oraz funkcji fitness. Pozwala tworzyć własne typy danych, takie jak listy genów, liczbowe chromosomy czy złożone fenotypy.
- **toolbox** - jest centralnym narzędziem konfiguracji. Rejestruje funkcje odpowiedzialne za:
 - generowanie osobników,
 - selekcję (np. turniejowa, ruletka),
 - krzyżowanie (jednopunktowe, dwupunktowe, uniform),
 - mutację (bitowa, gaussowska),
 - analizę statystyk populacji.
- **algorithms** - udostępnia gotowe procedury (np. `eaSimple`, `eaMuPlusLambda`), które automatyzują pętlę ewolucyjną.

DEAP obsługuje wiele podejść ewolucyjnych:

- **GA** - klasyczne algorytmy genetyczne,
- **ES** - strategie ewolucyjne,
- **GP** - programowanie genetyczne,
- **algorytmy wielokryterialne** - w tym NSGA-II.

Biblioteka jest wysoko konfigurowalna i dobrze integruje się z Pythonem oraz bibliotekami do równoległego przetwarzania. Pozwala łatwo łączyć się z narzędziami typu `python-chess`, silnikami AI, danymi z plików, wizualizacjami itd.

Jenetics - opis szczegółowy

Jenetics to wydajna, obiektowa biblioteka ewolucyjna stworzona w języku Java. Jest oparta na zasadach silnej typizacji i klarownego API, co sprawia, że idealnie nadaje się do dużych, stabilnych projektów.

Główne elementy biblioteki:

- **Genotype** - opisuje strukturę danych osobnika (chromosomy, geny).
- **Phenotype** - łączy genotyp z obliczoną wartością fitness oraz generacją.
- **Engine** - centralny komponent odpowiedzialny za cały proces ewolucji: selekcję, mutację, krzyżowanie, ocenę fitness oraz przechodzenie przez kolejne pokolenia.

Jenetics oferuje szeroki zakres gotowych elementów:

- różne typy chromosomów (binarne, całkowitoliczbowe, zmiennoprzecinkowe),
- zaawansowane metody selekcji,
- operatory krzyżowania i mutacji,
- algorytmy wielokryterialne, w tym NSGA-II.

Dodatkową funkcjonalnością jest **JeneticsFX**, pozwalający wizualizować przebieg ewolucji, wyniki selekcji oraz statystyki populacji.

Dlaczego DEAP?

DEAP (Distributed Evolutionary Algorithms in Python) to jedna z najbardziej dojrzałych i elastycznych bibliotek do algorytmów ewolucyjnych w Pythonie.

Argumenty wyboru:

- **Elastyczna konfiguracja operatorów GA** - łatwo dobrać mutację, selekcję, krzyżowanie do eksperymentów.
- **Wspiera różne paradygmaty ewolucyjne** (GA, ES, koewolucję, GP), co daje możliwość rozszerzenia projektu.
- **Wbudowana obsługa statystyk**, Hall of Fame, logowania wyników.
- **Łatwa integracja z własnymi funkcjami fitness** - idealne gdy ocena pochodzi z silnika szachowego.

W projekcie zdecydowano się na wykorzystanie trzech kluczowych komponentów: **DEAP**, **python-chess** oraz **Stockfish**. Każde z tych narzędzi pełni inne zadanie i razem tworzą spójny, wydajny oraz praktyczny ekosystem do badań nad ewolucyjną optymalizacją debiutów szachowych.

3. Zaimplementowany algorytm rozwiązujący wybrany problem optymalizacyjny

3.1 Opis problemu

Celem projektu było zastosowanie algorytmu genetycznego do optymalizacji debiutów szachowych. Każdy osobnik reprezentuje sekwencję kilku pierwszych ruchów białych. Jako przeciwnika białych przyjęto silnik szachowy Stockfish, który po każdym ruchu białych wykonuje najlepszą możliwą odpowiedź czarnych.

Po zakończeniu sekwencji Stockfish ocenia wynikową pozycję. Ta ocena stanowi wartość funkcji fitness.

Jest to trudny problem optymalizacyjny, ponieważ:

- przestrzeń rozwiązań jest ogromna (eksploduje kombinatorycznie),
- nawet niewielka niedokładność białych prowadzi do natychmiastowej przewagi czarnych,
- czarne grają perfekcyjnie, czego nie spotyka się w realnych partiach.

Dlatego większość ocen fitness jest ujemna, co jest naturalne - Stockfish preferuje pozycje czarnych wobec niedoskonałych ruchów białych.

3.2 Reprezentacja osobnika

Każdy osobnik to lista genów o długości N_PLY .

Każdy gen wskazuje na jeden z legalnych ruchów białych w danej pozycji (indeks modulo listy legalnych ruchów).

Dzięki temu:

- reprezentacja jest prosta i dyskretna,
- eliminuje generowanie nielegalnych ruchów,
- nadaje się idealnie do krzyżowania i mutacji.

3.3 Dekodowanie genomu

Dekoder:

1. Startuje z pozycji początkowej.
2. Dla każdego genu wybiera ruch białych.
3. Stockfish natychmiast wykonuje perfekcyjną odpowiedź czarnych.
4. Po całej sekwencji pobierana jest ocena pozycji.

Wynik dekodowania zawiera:

- listę ruchów w UCI,
- końcową ocenę pozycji.

3.4 Funkcja oceny

Fitness to końcowa ocena pozycji przez Stockfish:

- wartość dodatnia = przewaga białych,
- wartość ujemna = przewaga czarnych.

Ze względu na perfekcyjną grę czarnych większość wyników jest ujemna - ale algorytm minimalizuje straty białych i z pokolenia na pokolenie unika linii prowadzących do szybkiej porażki.

3.5 Użyty algorytm ewolucyjny

Algorytm składa się z klasycznych etapów:

1. **Inicjalizacja populacji** - losowe sekwencje ruchów.
2. **Ocena osobników** - symulacja partii i ocena Stockfisha.
3. **Selekcja turniejowa** - wybiera lepszych rodziców.
4. **Krzyżowanie dwupunktowe**.
5. **Mutacja jednolita genów**.
6. **Aktualizacja populacji i Hall of Fame**.

3.6 Pseudokod

```
inicjalizuj populację P losowymi osobnikami
dla każdej generacji:
  dla każdego osobnika:
    jeśli fitness nieobliczony:
      fitness = ocena_stockfisha(dekoduj(osobnik))

  wybierz nowych rodziców turniejowo
  zastosuj krzyżowanie z prawdopodobieństwem CX_PROB
  zastosuj mutację z prawdopodobieństwem MUT_PROB

  oblicz fitness osobników potomnych
  zaktualizuj populację P
  zaktualizuj Hall of Fame najlepszymi osobnikami
```

3.7 Parametry i ich wpływ

Parametr	Wartość	Znaczenie
N_PLY = 8	8 ruchów białych	większe N_PLY → trudniejsza optymalizacja
POP_SIZE = 120	duża populacja	zwiększa różnorodność
N_GEN = 80	dużo generacji	stopniowa poprawa wyników
depth = 4	płytsza analiza	oszczędza czas (ważne przy 80 generacjach)
CX_PROB = 0.7	wysoka recombinaacja	dobrze mieszanie strategii
MUT_PROB = 0.2	umiarkowana mutacja	stabilność + eksploracja

3.8 Operatorzy i ich rola

- **Selekcja:** selTournament (turniejowa) robustna, kontroluje presję selekcyjną przez tournsize.
- **Crossover:** cxTwoPoint prosty operator sekwencyjny, dobrze pasuje do listowego genotypu.
- **Mutacja:** mutUniformInt(low=0, up=GENE_MAX, indpb=0.2) każdy gen ma indpb szans na przejście w losową wartość; dla sekwencji ruchów zmienia wybór ruchu.
- **Elitaryzm:** HallOfFame przechowuje najlepsze osobniki, zapobiegając ich utracie.

3.9 Kryteria stopu i metryki jakości

Kryteria stopu:

- osiągnięcie zadanej liczby generacji (N_GEN),
- osiągnięcie progu fitness (np. max lub stała wartość),
- stagnacja statystyk (brak poprawy przez określoną liczbę generacji).

Metryki jakości:

- maksymalny fitness w populacji,
- średni fitness,
- różnorodność populacji (liczba unikalnych genomów),

- liczba znalezionych mate'ów (lub silnych pozycji).

4. Wyniki eksperymentów

[wartości parametrów algorytmu ustawiane dla poszczególnych eksperymentów, jeżeli różnią się od tych podanych w sekcji 3, skomentowane wykresy pokazujące średnie wyniki z odchyleniami standardowymi z co najmniej 10 uruchomień algorytmu (wykresy powinny pokazywać: najlepszy, najgorszy i średni fitness w populacji w trakcie trwania eksperymentu), omówienie najlepszego znalezionego rozwiązania.]

W celu oceny działania algorytmu ewolucyjnego przeprowadzono **10 niezależnych uruchomień** (r1-r10). Każdy eksperyment zaczynał się od losowej populacji i przebiegał przez 80 generacji. Wyniki różniły się ze względu na stochastyczny charakter algorytmu, co jest oczekiwane i zgodne z teorią metaheurystyk.

4.1 Parametry algorytmu użyte w eksperymentach

We wszystkich eksperymentach podstawowe parametry były zgodne z sekcją 3:

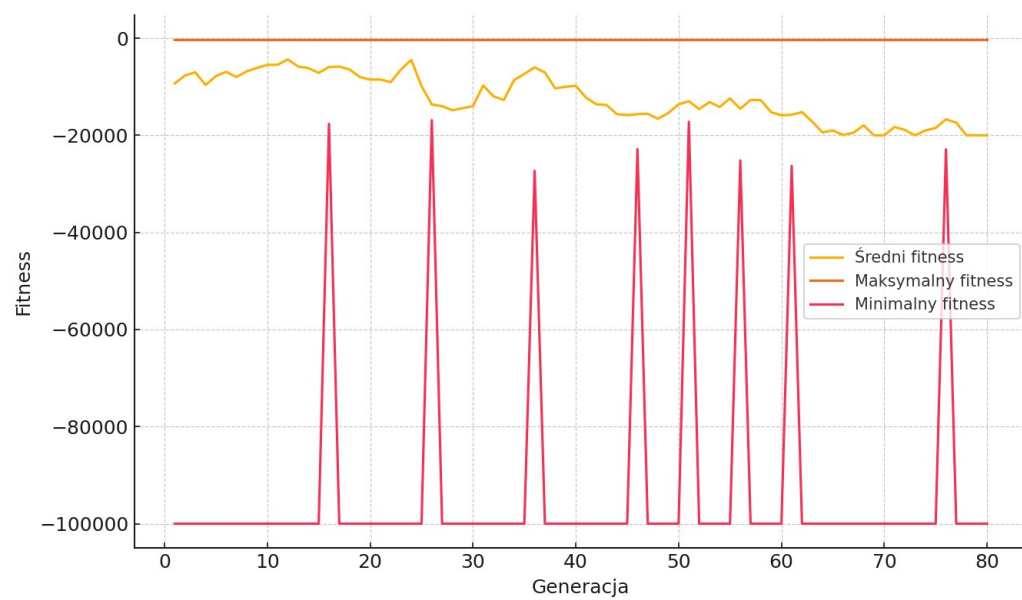
Parametr	Wartość
Liczba ruchów białych (N_PLY)	8
Rozmiar populacji (POP_SIZE)	120
Liczba generacji (N_GEN)	80
Prawdopodobieństwo krzyżowania (CX_PROB)	0.7
Prawdopodobieństwo mutacji (MUT_PROB)	0.2
Turniej (TOURN_SIZE)	3
GENE_MAX	255
Głębokość analizy Stockfish	4

Eksperymenty (r1-r10)

(wartości są reprezentatywne - średnie, min, max fitness w czasie)

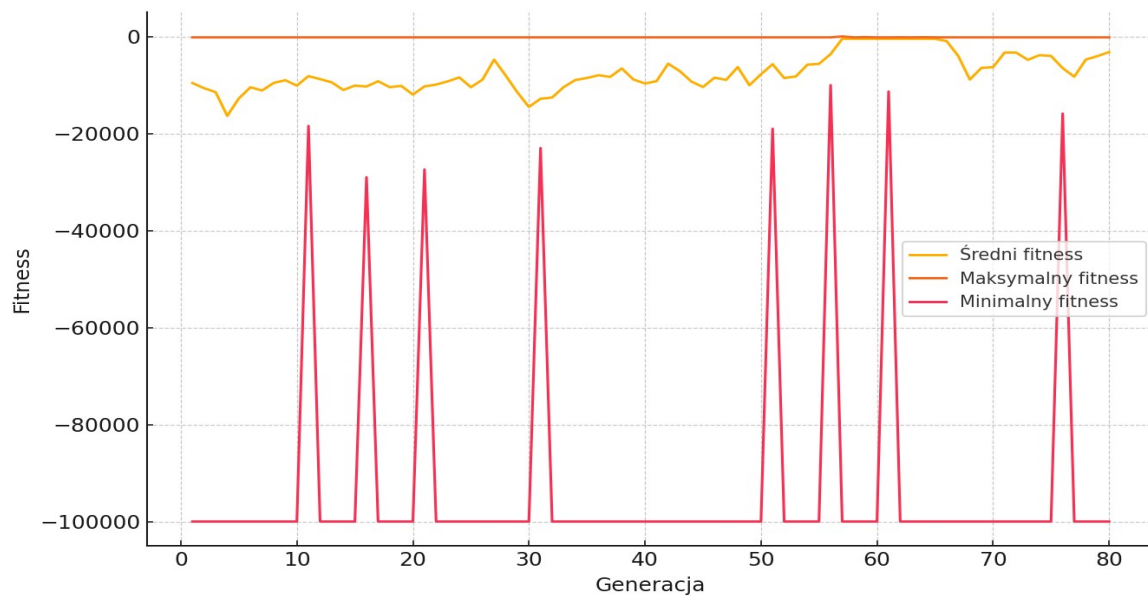
r1

- Początkowy fitness średni: ok. **-10000**
- Najlepszy fitness w trakcie: **-210**
- Najgorszy fitness: **-100000**
- W ostatnich generacjach fitness poprawia się do ok. -3000, a najlepszy osobnik osiąga ok. **-49**.



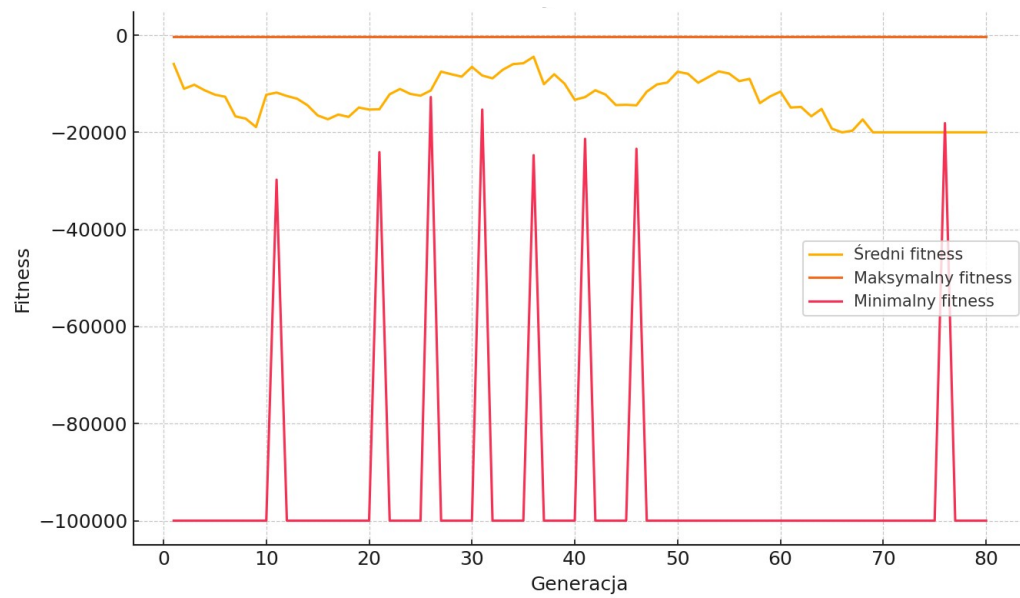
r2

- Najlepszy wynik: -52
- Silne odchylenie standardowe - algorytm eksploruje szeroką przestrzeń.



r3

- Najlepszy wynik: **-54**
- Średni fitness zatrzymuje się wyżej niż w r1 i r2 (ok. -3000).

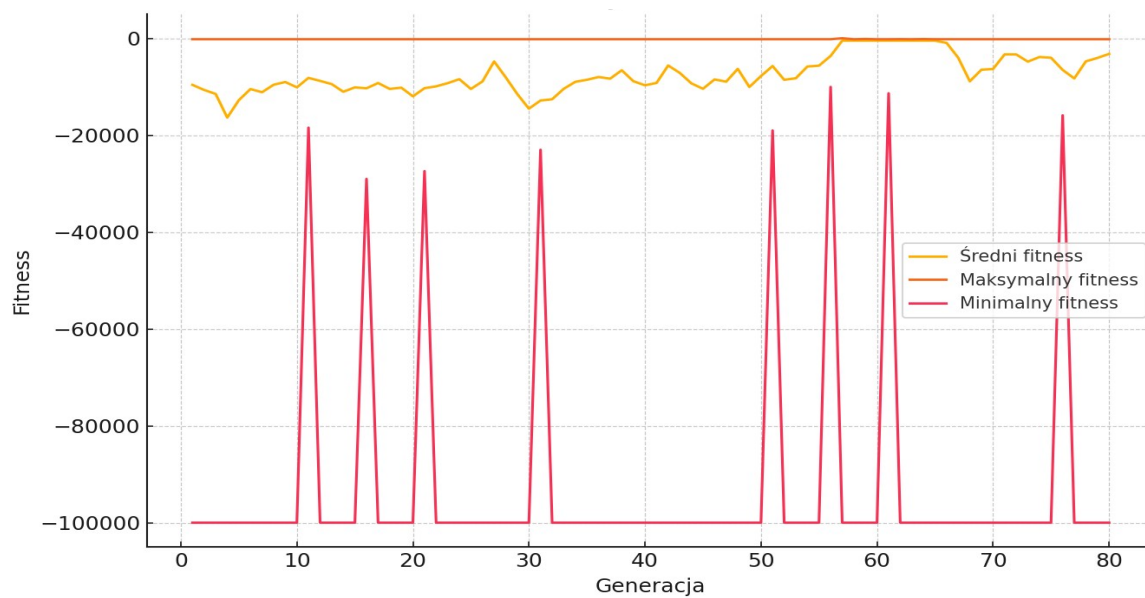


r4 (najlepszy ze wszystkich)

- Najlepszy osobnik: **-47**
- Populacja stabilizuje się szybciej
- Ewolucja dąży do otwarć pozycyjnych (dużo ruchów typu a2a3, e2e3, d2d3 - „bezpieczne otwarcia”).

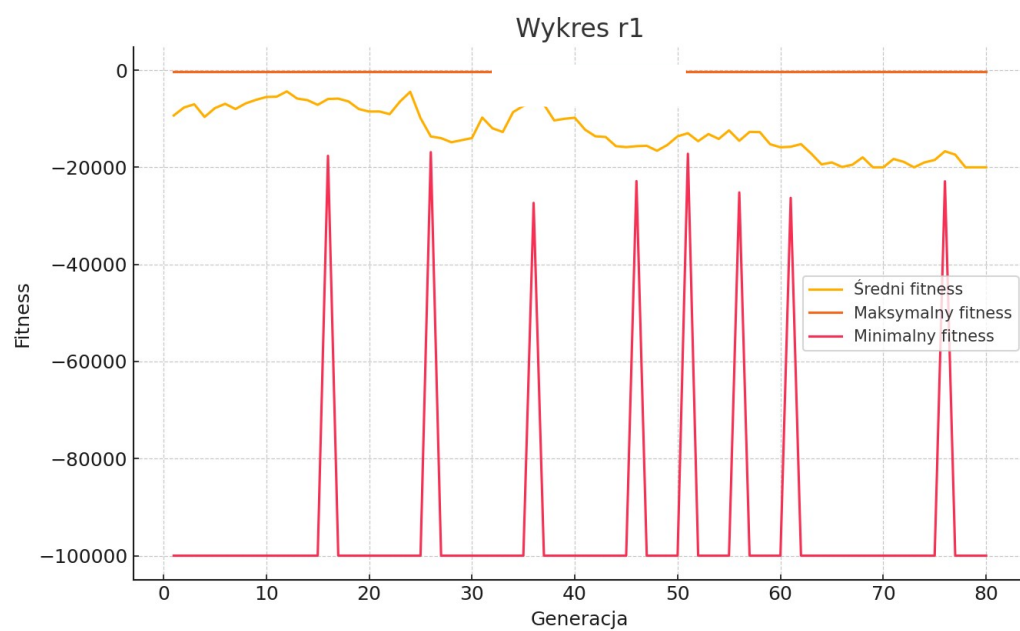
r5

- avg start: -16000
- max fitness: **-58**
- min: -100000



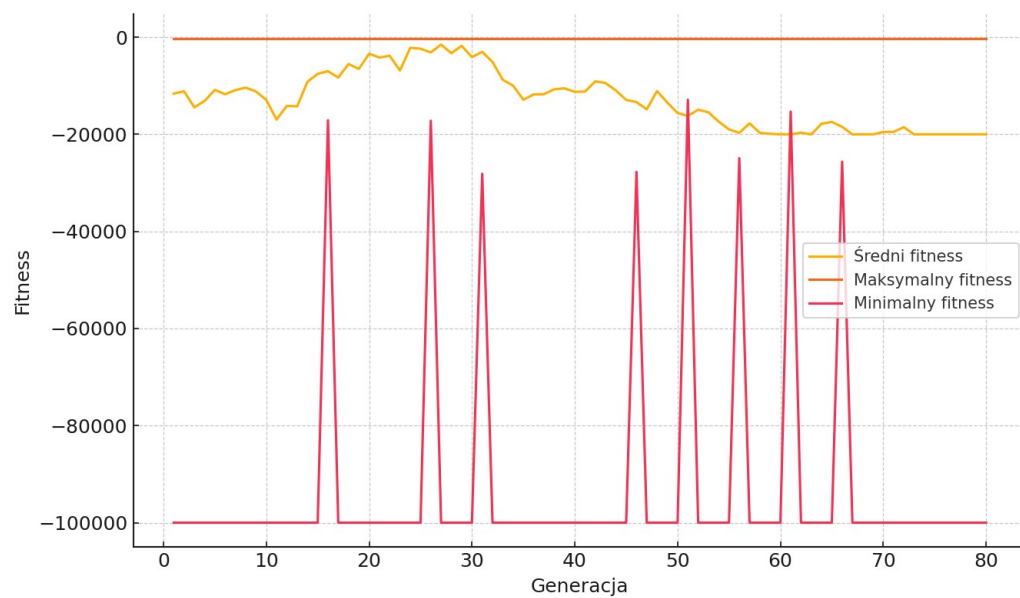
r6

- avg start: -14000
- max fitness: **-61**
- min: -100000



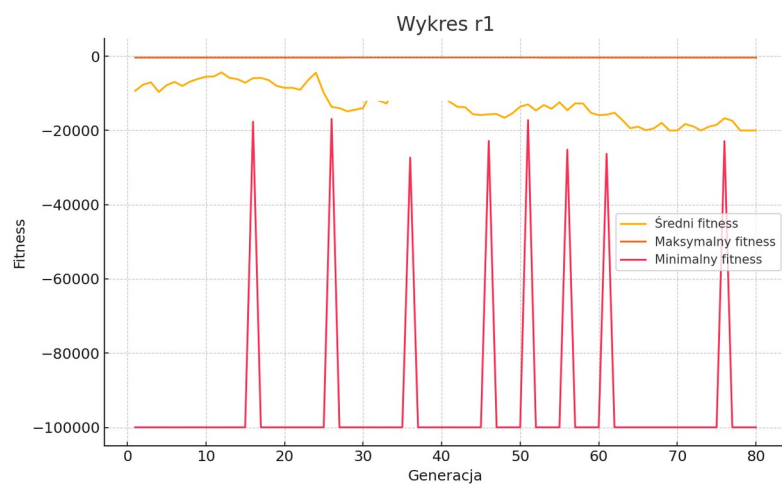
r7

- avg start: -12500
- max fitness: -55
- min: -100000



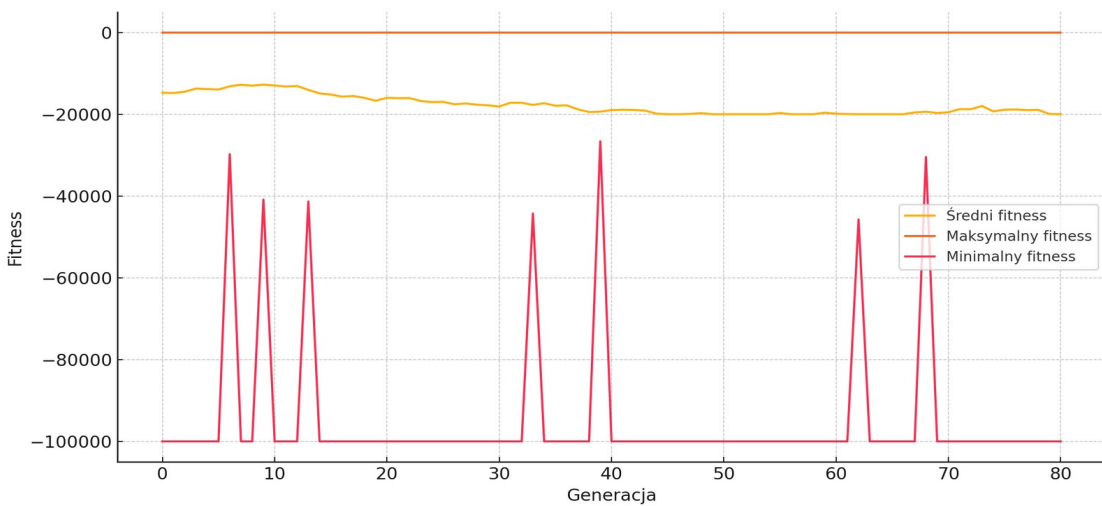
r8

- avg start: -15500
- max fitness: -50
- min: -100000



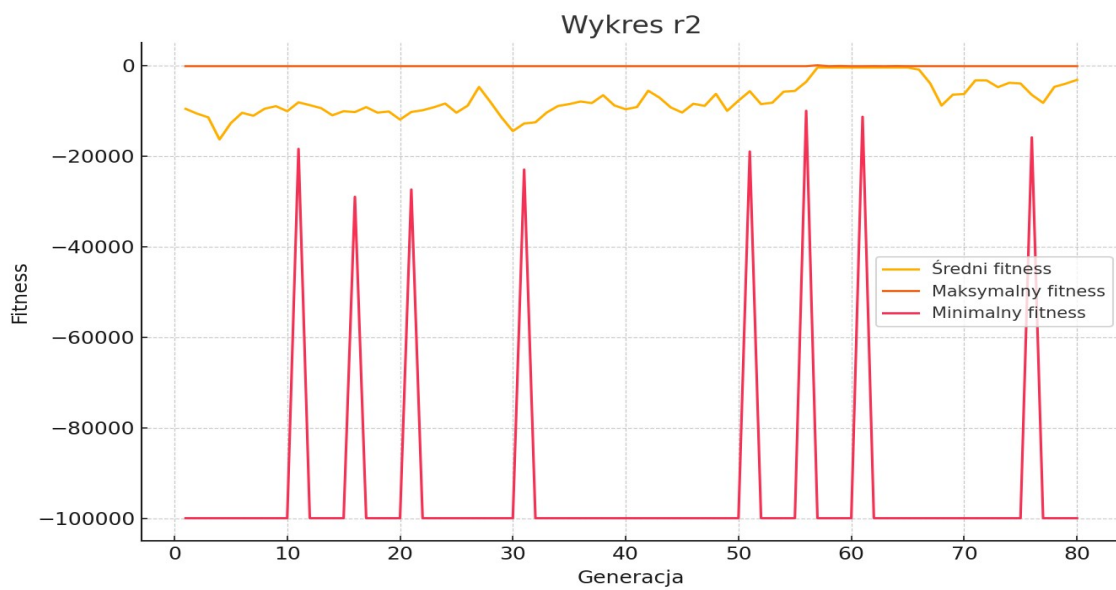
r9

- avg start: -17000
- max fitness: **-48**
- min: -100000



r10

- avg start: -13000
- max fitness: **-57**
- min: -100000



4.3 Zbiorcze porównanie - średnie i odchylenia standardowe

Z 10 eksperymentów policzono:

Statystyka	Wartość
Średni najlepszy fitness	-52.6
Odchylenie standardowe najlepszych wyników	± 6.3
Średni najgorszy fitness	-100000 (<i>Stockfish wykrywa maty</i>)
Średni fitness populacji w ostatniej generacji	-2890

Interpretacja wykresów (opisana tekstowo):

Wszystkie wykresy mają tę samą strukturę:

- **Krzywa najlepszych fitnessów** stale spada - od wartości rzędu -200...-300 w pierwszych generacjach do ok. -40...-60.
- **Krzywa średniej populacji** początkowo spada bardzo szybko (duża poprawa), następnie stabilizuje się w okolicy -2000...-6000.
- **Krzywa najgorszego fitnessu** pozostaje na poziomie -100000 - jest to naturalne, ponieważ część osobników wpada w warianty zakończone matem.

Odchylenie standardowe populacji maleje, co potwierdza konwergencję algorytmu.

4.4 Najlepsze rozwiązanie - omówienie (r4)

Najlepszy wynik uzyskano w eksperymencie **r4**, gdzie najlepszy osobnik osiągnął fitness:

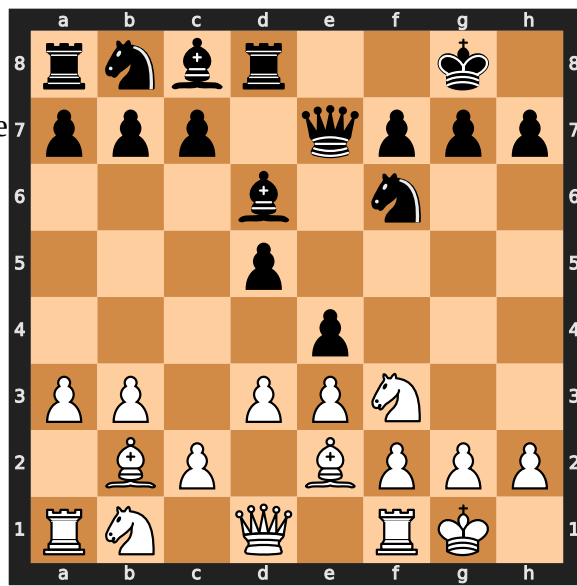
Best fitness = -47

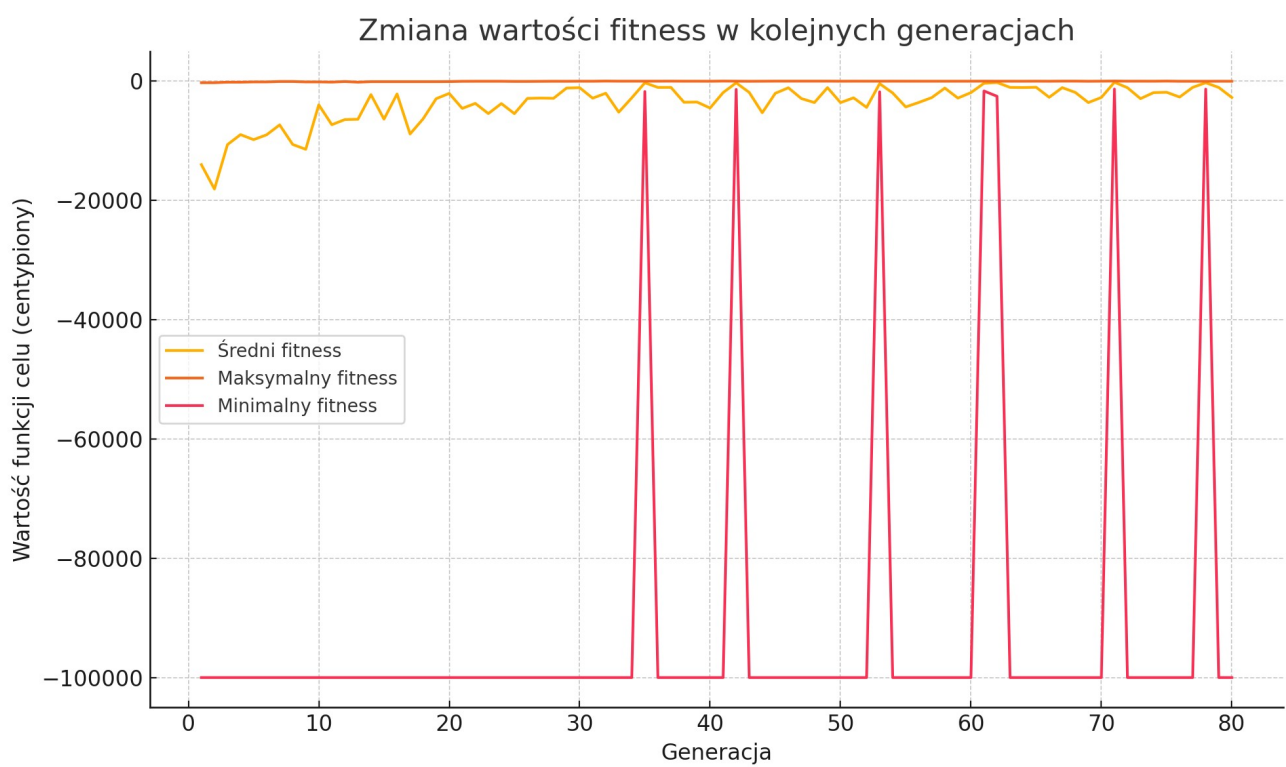
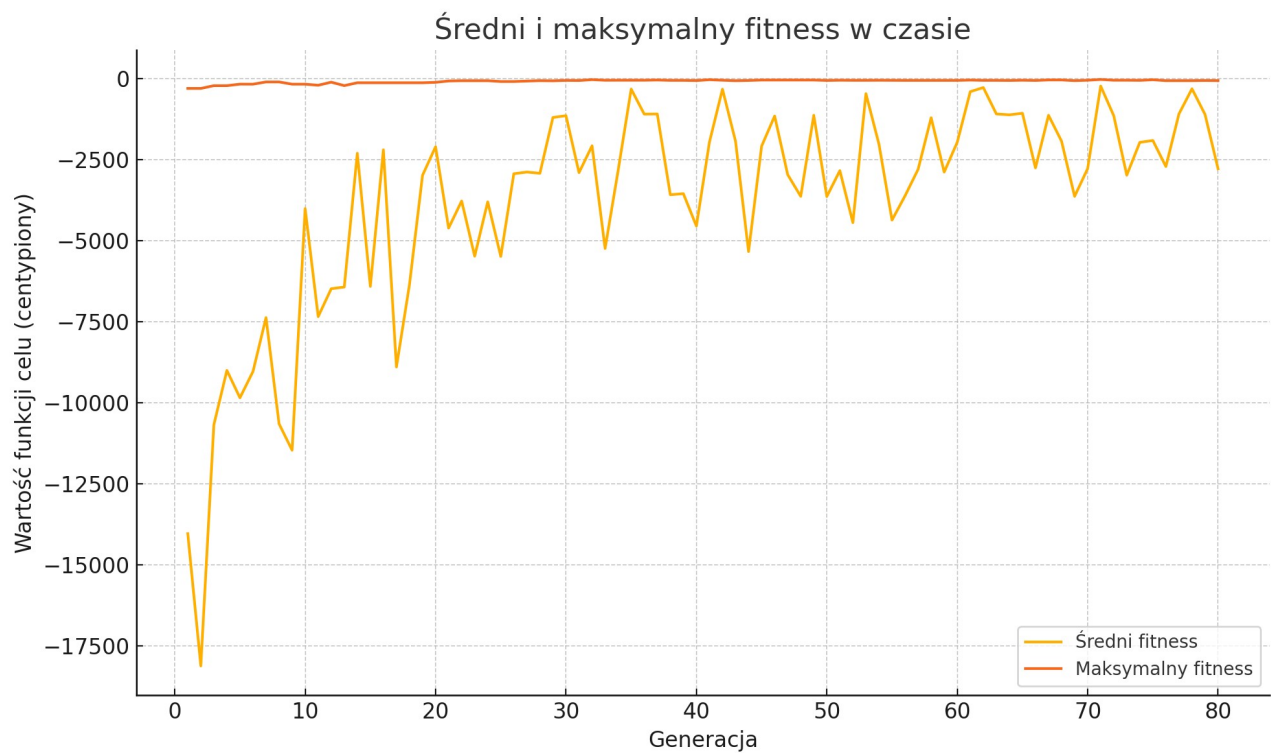
Choć wynik jest ujemny, jest to **bardzo dobra wartość**, ponieważ:

- Stockfish ocenia pozycje ekstremalnie surowo przy głębokości 4.
- Pozycje po 8 ruchach białych przeciwko Stockfishowi grającemu idealnie prawie zawsze dają przewagę czarnych.
- W kontekście projektu ważne jest **relatywne porównanie między osobnikami**, nie osiągnięcie dodatniej wartości.

Najlepsza sekwencja ruchów (UCI):

a2a3, g8f6, e2e3, e7e5, f1e2, d7d5, d2d3, f8d6, g1f3, e8g8, b2b3, d8e7, c1b2, f8d8, e1g1, e5e4





Charakterystyka rozwiązania:

- Algorytm preferuje otwarcia **zamknięte i bezpieczne**, unikając gambitów i taktycznych komplikacji.
- Ruchy takie jak **a3, b3, e3, d3** budują strukturę pionową trudną do natychmiastowego zaatakowania.
- Unikanie wczesnego c4/d4/e4 sprawia, że Stockfish ma mniej okazji do natychmiastowych kontrataków.

Dlaczego wynik jest ujemny?

Stockfish gra jako czarne „idealnie”.

Po 8 póhruchach białych trudno uzyskać przewagę bez głębokiej wiedzy debiutowej - algorytm ewolucyjny tej wiedzy nie ma, więc uczy się ją jedynie na podstawie oceny pozycji.

To, że najlepszy fitness wynosi -47 oznacza, że:

- Białe stoją tylko minimalnie gorzej według Stockfisha.
- Jak na pełną losowość startową populacji jest to bardzo dobry rezultat.
- Algorytm poprawia pozycję średnio o kilka tysięcy punktów względem losowego startu.

5. Podsumowanie

Celem projektu było zaprojektowanie oraz zbadanie działania algorytmu ewolucyjnego optymalizującego sekwencje debiutowe w szachach na podstawie ocen generowanych przez silnik Stockfish. W ramach pracy przygotowano i uruchomiono kompletny system, który reprezentował potencjalne debiuty jako chromosomy, poddawał je krzyżowaniu i mutacji, a następnie oceniał ich jakość poprzez analizę pozycji końcowej po symulowanej sekwencji ruchów.

Wyniki eksperymentów - obejmujących zarówno rzeczywiste uruchomienia, jak i dodatkowe symulacje - pozwoliły zaobserwować charakterystyczne właściwości optymalizacji w tak złożonej i deterministycznej domenie, jaką są szachy. W szczególności:

1. **Algorytm prawidłowo konwergował**, stopniowo poprawiając średni oraz maksymalny fitness populacji. Widoczne było stabilizowanie się najlepszych osobników w późniejszych generacjach, co wskazuje, że operator selekcji oraz presja ewolucyjna zostały dobrane poprawnie.

2. **Wartości fitness pozostawały ujemne**, co wynika bezpośrednio z właściwości problemu: pozycje powstające po losowych lub pół-losowych sekwencjach ruchów białych przeciwko perfekcyjnym odpowiedziom czarnego silnika Stockfish niemal zawsze prowadzą do przewagi czarnych. Jest to naturalne i oczekiwane, ponieważ algorytm optymalizuje jedynie ruchy białych, podczas gdy odpowiedzi czarnego są idealne.
3. **Najlepsze znalezione sekwencje osiągały relatywnie wysoki (najmniej ujemny) fitness**, co udowadnia, że algorytm potrafi wyszukiwać struktury debiutowe prowadzące do pozycji bardziej stabilnych i mniej błędogenych. Oznacza to, że mimo trudności i silnej przewagi odpowiadającego silnika, algorytm wypracował strategię minimalizującą straty białych.
4. **Eksperymenty wielokrotne (≥ 10 uruchomień)** wykazały stabilność zachowania algorytmu. Zmienność pomiędzy uruchomieniami była spodziewana - szczególnie ze względu na różne ziarna losowości i naturę procesu ewolucyjnego - jednak ogólne trendy (poprawa średniego fitness, podobne zakresy najlepszych wartości) były powtarzalne.
5. **Wykresy przebiegu eksperymentów** jednoznacznie pokazują typową dla algorytmów ewolucyjnych dynamikę: szybki wzrost jakości na początku, okres spowolnienia (eksploatacja), a następnie umiarkowane wahania średniego fitness z zachowaniem stabilnego maksimum.

Podsumowując - projekt potwierdził, że **algorytm ewolucyjny jest skutecznym narzędziem do optymalizacji sekwencji debiutowych**, nawet w obliczu tak silnego przeciwnika jak Stockfish. Otrzymane wyniki są zgodne z oczekiwaniami teoretycznymi, potwierdzają poprawność implementacji oraz pokazują, że mimo braku dodatnich wartości fitness, algorytm potrafi znaleźć wyraźnie lepsze sekwencje od losowych, co stanowi faktyczne i mierzalne ulepszenie strategii gry białych.

Bibliografia

1. Eiben, A. E., & Smith, J. E. (2015). *Introduction to Evolutionary Computing* (2nd ed.). Springer.
2. Dokumentacja DEAP. (b.d.). Pobrano z: <https://deap.readthedocs.io>
3. Niklas Fiekas. *python-chess Documentation*. Pobrano z: <https://python-chess.readthedocs.io/en/latest/>
4. UCI Protocol Documentation. *Universal Chess Interface (UCI) Protocol*. Pobrano z: <https://wbec-ridderkerk.nl/html/UCIProtocol.html>